Vyuha: A Layered, Defense-in-Depth System for LLM Jailbreak and Prompt-Injection Defense

U E Sai Pavan Vamshi Krishna (G25AIT2149) - M.Tech, Indian Institute of Technology Jodhpur Course project, CSL6010 (Cyber Security). Successor to **RJD-v2**. 2026.

(Vyuha was formerly named "Aegis"; it was renamed to avoid a name collision with NVIDIA's "Aegis" content-safety guard. "RJD-v2" remains the name of the fast L1 detector.)

Abstract

Large language models deployed as assistants and autonomous agents face a moving adversary: jailbreaks that strip safety behaviour, prompt injections hidden in retrieved content, obfuscated and translated attacks, semantic (persuasion) attacks that carry no surface tell, and responses that leak secrets or private data. Prompt injection is the **#1 risk** in the OWASP Top 10 for LLM Applications, and recent adaptive-attack work shows that *any single* filter, once known, can be evaded - "the attacker moves second." We present **Vyuha**, a defense-in-depth system that composes six independent layers (L0-L5): input normalization and provenance, a fast statistical detector, a content guard, agent/tool-injection defense, output moderation, and continuous red-teaming with a self-hardening loop. The contribution is not any single classifier but the **composition**, its **reproducibility on a single free GPU**, and three measured claims: (C1) robustness to *compositional* obfuscation, (C2) an explicit jailbreak-vs-prompt-injection separation, and (C3) a free-compute self-hardening loop. We evaluate along three attack axes, each mapped to the layer that owns it. On **obfuscation**, the fast detector RJD-v2 reaches recall 1.00 on Base64, ROT13, character-spacing and a *held-out* wider-spacing variant, matching a GPU DeBERTa guard's robustness at ~8 ms on CPU with lower over-refusal (FRR 0.044 vs 0.113). On **harmful-topic** and **semantic** attacks, the fast layer is - by design - weak (it flags <7% of real PAIR jailbreaks), so a composed content guard (Qwen3Guard-0.6B) carries those axes (90% detection of PAIR at 4.8% false-positives), inflating the adaptive attacker's estimated query cost ~10x. We report security-grade metrics, name our limitations honestly (our own tuned guard is jailbreak-only; our agent layer is behind capability-based defenses such as CaMeL), and release everything as an open-source library, a service, and reproducible notebooks.

1. Introduction

Safety-aligned LLMs still fail under adversarial pressure. A *jailbreak* is a prompt that induces a model to violate its safety policy ("ignore all previous instructions and act as DAN"); a *prompt injection* embeds adversarial instructions in data the model later reads (a web page, a tool result, an email), hijacking an agent without the user's knowledge. Both are industrialized: prompt-sharing communities circulate thousands of working jailbreaks, and agentic deployments multiply the blast radius because a single injected instruction can trigger real-world tool calls and data exfiltration.

The central difficulty is adaptivity. A classifier that blocks today's attacks is a fixed target; attackers paraphrase, encode (Base64, leetspeak), substitute homoglyphs, insert zero-width characters, *compose* several of these, translate to other languages, or - most subtly - use fluent persuasion that contains no keyword or obfuscation tell at all. The "attacker-moves-second" result is that robustness claims based on a static test set systematically overstate real protection. The practical response - adopted by every serious security discipline - is **defense in depth**: many cheap, independent controls layered so an attacker must defeat all of them, plus a *continuous* red-team/monitoring loop so the controls keep pace.

Vyuha operationalizes this for LLMs under a hard constraint: it must be buildable and reproducible on free compute (a single Kaggle T4). This forces dependency-light designs and graceful offline fallbacks, which also make the system easy to adopt. Crucially, we do not claim novelty at any single layer; strong solutions already exist for each. Our contribution is the honest *composition* and a benchmark-backed division of labour across attack axes.

2. Threat Model (2026)

We defend against an adversary who controls the prompt and/or any untrusted content the model ingests, and who can adapt to a known filter. In scope:

- **Direct jailbreaks:** persona/role-play ("DAN", "developer mode"), instruction-override, policy-nullification.
- **Obfuscation/evasion, including compositions:** Base64 and other encodings, leetspeak, homoglyphs, zero-width and bidi control characters, emoji/variation-selector smuggling, character-spacing, full-width unicode, and *stacked* combinations (e.g. zero-width + character-spacing).
- **Semantic attacks:** meaning-preserving reformulations with no surface tell - iterative refinement (PAIR, TAP) and persuasion (PAP).
- **Indirect / agent injection:** instructions hidden in retrieved documents, web pages, or tool outputs that target an agent and its tools (exfiltration, unauthorized actions).
- **Multilingual attacks** and **output-side failures** (PII, leaked secrets, system-prompt leakage, harmful compliance).

Out of scope: attacks on the base-model weights, host infrastructure, or human operator. We assume no defense is unbreakable; the goal is to **raise attacker cost** and **shrink the attack surface** of every channel.

3. Related Work

Guard models. Instruction-tuned safety classifiers - Llama Guard (through v4), **Qwen3Guard**, ShieldGemma, WildGuard, NVIDIA **Aegis** Content Safety, and GuardReasoner - judge prompts and (prompt, response) pairs against a taxonomy. They are strong but heavyweight, GPU-bound, and themselves evadable; recent work shows evasion of six commercial guardrails, and notes that *none of them separate jailbreak from prompt-injection classes*.

Adaptive & semantic attacks. PAIR jailbreaks in <20 queries; TAP adds tree search with pruning; PAP weaves 40 social-science persuasion strategies into human-readable prompts (~92% ASR on aligned models). "The Attacker Moves Second" breaks 12 defenses at >90% ASR - motivating our continuous-red-team stance.

Obfuscation defenses. Input normalization / decode-and-rescreen is a known pattern (DecipherGuard; Broken-Token's characters-per-token filter; perplexity filters), but the literature observes that *compositional* encodings still leak through single-transform normalizers - the gap our L0 targets.

Agent defenses. The **Dual-LLM / quarantine** pattern (Willison) and Google DeepMind's **CaMeL** (a privileged planner plus a capability-tracking interpreter, ~67% of AgentDojo injections mitigated, the first with strong guarantees) are the state of the art; **spotlighting** marks untrusted text; **StruQ/SecAlign** are training-time defenses.

Lifelong / adaptive guardrails. AGrail (ACL 2025) generates and continually optimizes agent safety checks; **JBShield** (USENIX Sec 2025) detects and *steers* jailbreaks via activated-concept analysis in the model's hidden states (white-box).

Benchmarks, red-teaming, data protection, standards. HarmBench, JailbreakBench, AdvBench, StrongREJECT, AgentDojo/InjecAgent; garak, PyRIT, Promptfoo; Presidio and the secret-scanning rule families; the **OWASP LLM Top 10** and **NIST AI RMF**.

Vyuha does not replace these; it **composes** their ideas into one free-compute cascade, and positions each layer honestly against the strongest existing work (a content guard for semantics, CaMeL for agents).

4. System Design

Vyuha is a cascade of six independent, individually-testable layers.

L0 - Normalize & provenance. `normalize()` performs NFKC folding; strips zero-width, bidi, tag and variation-selector characters; folds homoglyphs; decodes Base64/ROT13; de-leetspeaks; and de-spaces with an **adaptive character-gap** rule that survives composition with zero-width injection. Each recovered form is exposed as a separate **view**; downstream scoring takes the **max over views**, so a long obfuscation cannot dilute a short recovered instruction. `spotlight()` wraps untrusted content in explicit data markers.

L1 - Fast detector (RJD-v2). The shipped L1 is **RJD-v2**: de-obfuscation-normalized word/char TF-IDF plus 13 engineered features, adversarially augmented and calibrated, CPU-only (~8 ms). We also implemented a `FastLayer` that adds a semantic-similarity signal and a signature database via a recall-preserving max; our evaluation (Section 6) shows this ensemble *raises over-refusal without earning its keep*, so **RJD-v2 is the default L1** and the extra signals are optional. This is a deliberate, measured design decision, not an omission.

L2 - Content guard. For harmful-topic and semantic coverage the content guard is **Qwen3Guard-0.6B** (Apache-2.0, T4-friendly), invoked by a **selective cascade** on the L1-uncertain band, or on every input in a higher-assurance mode. We also fine-tune

a small QLoRA guard (Qwen2.5-1.5B) and publish it; Section 6 shows it is a *jailbreak-specific* guard (strong out-of-distribution on jailbreaks, inert on harmful content), so it complements rather than replaces the content guard.

L3 - Agent defense. An `InjectionScanner` (normalize-first) detects and sanitizes injected instructions in untrusted content; a least-privilege `ToolPolicy` tracks taint and gates dangerous tools once a turn reads untrusted data; a `DualLLM` orchestrator keeps the privileged planner from ever seeing raw untrusted text. This realizes the Dual-LLM pattern on free compute; it is weaker than CaMeL's capability guarantees, which we adopt as the target.

L4 - Output moderation. `OutputModerator` is the egress gate: redact **PII**, block leaked **secrets/credentials**, detect **system-prompt/canary** leakage, and score **response safety**. Response-harm is scored by a *content* guard (Qwen3Guard) judging the **(prompt, response) pair** through `proba_response` - i.e. whether the model actually *complied* with something harmful - not the L1 jailbreak detector, which scores how jailbreak-like a string looks and is the wrong signal for a fluent, harmful answer.

L5 - Continuous ops + self-hardening. A `RedTeam` harness mutates known attacks through single and **pairwise-composed** evasions and reports ASR per mutator; a `Monitor` flags drift via the Population Stability Index. The **self-hardening loop** is a runnable `SelfHardeningLoop` component: it red-teams the detector, harvests the attacks that still evade, auto-generates a normalized-key signature for each (pure re-encodings collapse onto one key), folds it into the scorer, and re-measures - under an **FRR budget** that rolls back any round which raises over-refusal. It measures a **held-out novel-attack set** every round, so the honest limit (signatures harden *known* attacks, not novel ones) is reported rather than hidden. Measured in Section 6.

Pipeline. `Vyuha.scan()` runs L0->L1->(L2); `Vyuha.guard_turn(prompt, response)` adds the L4 egress gate.

5. Implementation

A single Python package with a light core (scikit-learn, pandas, numpy); embedding/guard/serving are optional extras; every layer has an offline fallback. Vyuha ships as the `vyuha-guard` `pip` library with a CLI, a **FastAPI** service (`/scan`, `/moderate`, `/guard_turn`) containerized via Docker, and Kaggle notebooks that clone the repo and reproduce each phase on a T4, logging to Weights & Biases.

6. Evaluation

Methodology. We report **security-grade** metrics - ROC-AUC, **recall @ 1% FPR**, **over-refusal (FRR)**, end-to-end **attack-success-rate (ASR)** under graded judges, semantic-attack detection, and latency - not plain accuracy. The corpus assembles in-the-wild jailbreaks, JailbreakBench, AdvBench, HarmBench and WildGuardMix; benchmark sets are held test-only. We evaluate three attack axes.

(a) Obfuscation (L0/L1). RJD-v2, in-distribution (n=1605): **ROC-AUC 0.923, recall@1%FPR 0.330, FRR 0.044**, ~8 ms CPU - versus the public DeBERTa guard

(`deberta-v3-base-prompt-injection-v2`) at 0.896 / 0.210 / FRR 0.113, ~62 ms GPU. Recall under attack is **1.00** on Base64, ROT13, character-spacing, and a **held-out** wider-spacing variant the detector never trained on; leetspeak 0.966. Character-spacing was the prior open gap (ASR 0.83); a multi-view-max + adaptive-gap de-spacing fix closed it to **0.00**, and the fix *generalizes* to the held-out variant - our C1 evidence. The full `FastLayer` ensemble scores 0.875 / FRR 0.175 (worse than RJD-v2), because its signature templates false-fire on benign text; hence RJD-v2 ships as L1.

(b) Harmful-topic (L2). On StrongREJECT (313 prompts, fine-tuned judge) the victim is already largely safe; the input filter cleanly blocks the encoding channel (Base64 313/313) but does not reduce bare harmful-topic ASR - by design, L1 detects jailbreak *patterns*, not harmful *content*. Over-refusal on XSTest is **0.008** for RJD-v2. The content guard carries the topic axis: Qwen3Guard flags **79%** of XSTest's unsafe half (vs ~1% for the L1 detectors) at 4.8% over-refusal, and in a guard-on-everything ensemble drops char-spaced / zero-width harmful ASR by 83-92% (n=60).

(c) Semantic (L2). On **103 real PAIR jailbreaks** (JailbreakBench artifacts) the L1 detectors flag only **6.8%** (no surface tell) and our tuned jailbreak guard only 2.9% - but the content guard **Qwen3Guard flags 90.3%** at 4.8% false-positives. Attack efficiency: undefended PAIR jailbreaks the target in a **median 30 / mean 51** queries; behind the content guard only 9.7% evade, an *estimated* **~10x** cost inflation (~300-500 queries per success). This is the concrete, measured answer to "do semantic attacks defeat L1, and what carries them?" - they do; L2 does.

Guard generalization (L2, our QLoRA guard). Cross-benchmark ROC-AUC **0.72-0.92** on jailbreaks it never trained on, at FRR 0.03-0.06 - but 0.000 on XSTest harmful and 0.029 on PAIR. It is a *jailbreak* guard (better OOD jailbreak generalization than the L1 classifier), not a content guard.

Upper layers. L3: indirect-injection detection **1.00 vs 0.00-0.17** for a regex-only baseline across Base64/homoglyph/zero-width/character-spacing, at benign-pass 1.00. L4: precision = recall = **1.00** on the labeled leak/harm probe; and with response-harm scoring moved to the content guard, fluent **cue-less** harmful compliances that the keyword heuristic misses entirely are caught - on a small contrast set (illustrative, n=5) Qwen3Guard on the (prompt, response) pair scores **F1 1.00** versus **0.00** for the heuristic. L5: on a detector with a deliberately exhibited obfuscation gap (RJD-v2 itself has few, so it mostly reports "converged"), the runnable self-hardening loop drives seen-attack **ASR 0.62 -> 0.00 in one round at flat FRR 0.000**, while held-out *novel* attacks stay at **0.88** (signatures harden known attacks, not novel ones - reported, not hidden); two earlier hand-run cycles are on record (char-spacing 0.83 -> 0.00; adaptive ASR 1.00 -> 0.50 as the surviving evasions shifted from obfuscation to injection-wrappers), and the PSI drift monitor trips (**PSI 11.8**) on an attack-surge window while staying quiet on normal traffic. Point estimates depend on the run and gated-dataset access; the notebooks reproduce them end to end.

7. Limitations

No single defense - and no stack - is unbreakable; Vyuha raises cost, it does not guarantee safety. Concretely and honestly: (i) **L1 is a surface/pattern detector** - it does

not and should not catch harmful-topic or semantic attacks (6.8% on PAIR); those depend entirely on the L2 content guard. (ii) **Our own tuned guard is jailbreak-only**; harmful-topic and semantic coverage comes from composing an open content guard (Qwen3Guard), which we do not claim to have improved upon. (iii) **The agent layer (L3) is behind** capability-based defenses such as CaMeL - it is a lightweight, black-box approximation without provenance guarantees, and a fair AgentDojo comparison needs an LLM backend beyond free compute. (iv) The defended attack-efficiency figure is an **estimate** under the measured detection rate, not a direct "PAIR-against-Vyuha" measurement. (v) Secret/PII regexes trade recall for precision; offline fallbacks reduce accuracy; the red-team mutates *known* attacks. (vi) The obfuscation-robustness advantage is **shared** with strong guardrails (protectai is also robust); our defensible edge is cost + reproducibility + composition robustness + held-out generalization, not obfuscation alone. These trade-offs are exactly why L5 (continuous red-teaming + self-hardening) exists.

8. Ethics and Responsible Use

Vyuha is a **defensive** safety filter. The attack and red-team code mutate only attacks already present in public corpora, for evaluation and hardening; the repository contains no novel weaponization or operational instructions for harm. Semantic-attack prompts are loaded read-only from the public JailbreakBench artifacts for measurement. The system is aligned to the OWASP LLM Top 10 and NIST AI RMF and is intended to be operated with continuous red-teaming and human oversight. Detection scores are probabilistic and should inform, not solely determine, high-stakes decisions.

9. Conclusion

Vyuha shows that a credible, modern LLM defense can be assembled from independent, individually-cheap layers - entirely on free compute - and packaged as a usable library and service. The contribution is the **composition** and its honest, benchmark-backed division of labour across three attack axes: obfuscation is closed cheaply at L0/L1 (RJD-v2, recall 1.00 incl. a held-out variant, 8 ms CPU, matching a GPU guard at lower over-refusal); harmful-topic and semantic attacks - which by design defeat a surface detector - are carried by a composed content guard (Qwen3Guard, 79% / 90%); and a self-hardening red-team loop keeps pace with the adaptive attacker, raising estimated attack cost ~10x. We are explicit about where existing work is stronger (content guards for semantics, CaMeL for agents) and about which of our own components did not earn their place (the L1 ensemble extras, our jailbreak-only tuned guard). Future work: capability-based agent defense (CaMeL-style), a calibrated content guard for the response gate, a direct attack-efficiency measurement, and native multilingual guard training.

References (selected)

OWASP Top 10 for LLM Applications (LLM01). · NIST AI RMF 1.0. · Inan et al., *Llama Guard*. · *Qwen3Guard*; *ShieldGemma*; *WildGuard*; NVIDIA *Aegis Content Safety*; *GuardReasoner*. · Willison, *Dual-LLM*; DeepMind, *CaMeL*. · Luo et al., *AGrail* (ACL

2025). · Zhang et al., *JBShield* (USENIX Sec 2025). · Chao et al., *PAIR / JailbreakBench*. · Mehrotra et al., *TAP*. · Zeng et al., *PAP (persuasion)*. · Souly et al., *StrongREJECT*. · Mazeika et al., *HarmBench*. · Zou et al., *AdvBench*. · Debenedetti et al., *AgentDojo*; *InjecAgent*. · *Broken-Token*; *DecipherGuard*. · Chen et al., *StruQ*; *SecAlign*. · Hines et al., *Spotlighting*. · NVIDIA *garak*; Microsoft *PyRIT*; *Promptfoo*. · Microsoft *Presidio*. · "The Attacker Moves Second."